Supplementary material for "LunarAtlas: A Reproducible Data Processing and Interactive Visualization Infrastructure for Chandrayaan-3 LIBS Level-1 Archives"

Lovekesh Anand, Dua Saeed

Every number is produced by the scripts and settings in `revision_v2/` of the repository (see `revision_v2/README.md`); machine-readable tables are in `revision_v2/outputs/`. References to main-text figures, tables and sections use the numbering of the manuscript.

S1 Archive audit: counts, pairing intervals, determinism and migration

Earlier counts. A previous version reported 779 ingested measurements. That figure came from an earlier processing version that stored the 132 unpaired plasma acquisitions without subtraction and 34 background frames from the 33 background-only products as cleaned measurements ($613 + 132 + 34 = 779$). The current version stores only the 613 paired acquisitions as cleaned. Raw records are ingested for the 230 products that have cleaned output (1,125 acquisitions, including 14 unpaired plasma acquisitions); the 156 acquisitions of the 71 products without any paired plasma remain in the archived files only. The public deployment has not yet been re-ingested with the corrected version. A reversible migration (build the corrected database separately, copy the application tables, swap by renaming, roll back by renaming back) was rehearsed on a disposable copy reconstructed with the legacy code (`revision_v2/rehearse_migration.py`); it has not been applied to the deployment.

Pairing intervals. Label timestamps have 1 s resolution. The paired background carries the same second as the plasma acquisition for 206 acquisitions, the previous second for 264, and 2–13 s earlier for 143 (consecutive plasma acquisitions reusing one background). The assumption of a stable background therefore spans up to 13 s for some pairs and cannot be checked more finely from the labels.

Determinism and ingestion. Running stage 2 twice on the same input produced byte-identical cleaned spectra, parameter files and label copies for all 230 products (per-run metadata files differ only by their processing timestamp). Ingestion into a disposable PostgreSQL 18 database (`revision_v2/run_isolated_ingestion.py`) stored 613 cleaned acquisitions with 2,094 points each (1,283,622 points), matching the processed files measurement by measurement (point counts and intensity sums identical); a second ingestion of the same files added no rows to any table, and no (measurement, wavelength) pair occurs twice. For every cleaned point, $\max(0, I_P - I_B)$ recomputed in SQL from the stored raw plasma and the stored paired-background row differs from the stored value by 0.0, and the pairing and clip-fraction fields were present for all 613 records. Deterministic processing and idempotent ingestion are distinct properties and were tested separately.

S2 Worked example: window statistics, noise propagation and clipping

Over 560–580 nm (58 channels) the raw plasma acquisition of ID 3 has mean 1,387.4 a.u. (std 54.0) and the paired background 943.3 a.u. (std 50.6); the cleaned spectrum has mean 444.1 a.u. (std 70.8), no negative residuals in this window, and a mean ratio raw/cleaned of 3.12 (Table S1). Over all 2,094 channels 0.48% of residuals were negative before clamping (0% in 220–800 nm). The ratio of standard deviations, cleaned/raw, is 1.31. For two independent inputs with these unequal variances the expected ratio would be $\sqrt{s_P^2 + s_B^2}/s_P = 1.37$ (not $\sqrt{2}$); the sample covariance of I_P and I_B across the window ($r = +0.085$) accounts exactly for the difference, as it must algebraically. Because the window standard deviation mixes detector noise with

channel-to-channel spectral structure, and the window has only 58 channels, these statistics are descriptive; they neither demonstrate correlated CCD readout nor independently verify the subtraction.

ID	$\bar{I}_P$	$\bar{I}_{\text{clean}}$	s_{clean}/s_P	Neg. (%)
1	1,005.3	69.9	1.25	15.5
2	983.3	12.3	0.47	74.1
3	1,387.4	444.1	1.31	0.0
4	1,194.6	38.9	0.92	32.8

Table S1: 560–580 nm window statistics (a.u.); Neg.: residuals < 0 before clamping in the window. Clamping lowers s_{clean} where many residuals are negative.

ID	Mean	Std	Max	Clip (%)	Clip cal. (%)
1	61.0	56.9	321	22.5	22.3
2	9.4	23.8	261	76.8	77.6
3	416.9	95.0	684	0.5	0.0
4	54.1	62.4	637	28.1	27.9

Table S2: Cleaned intensity (a.u., 2,094 channels) and clip fraction (negative residuals before clamping) over 2,094 channels and over the 1,684 calibrated-range channels.

Clip fraction across the archive. Of the 613 paired acquisitions, 256 (41.8%) exceed a clip fraction of 0.5 over 2,094 channels (251 over the 1,684 calibrated-range channels); at 0.25 and 0.75 the counts are 372 (60.7%) and 157 (25.6%) (main-text Figure 3). The distribution is broad, with modes near 0 and 1, so the flagged count depends strongly on the threshold; 0.5 is a convention, not an optimised or physically validated cut-off. The clip fraction is strongly anti-correlated with the mean cleaned intensity (Spearman $\rho = -0.93$), which is expected because both are functions of the same residual, and only weakly with the number of detected peaks ($\rho = -0.28$) and a robust peak signal-to-noise ratio ($\rho = -0.15$). These relationships are descriptive and do not establish which spectra are scientifically usable. All 613 acquisitions, flagged or not, are stored. The clip fraction, the paired-background row and the subtraction flag are stored in `measurement_clean` and in each product's `_params.csv` file. No API endpoint returns them yet: the response schemas of `/api/v1/measurements`, `/api/v1/public/measurements`, `/api/v1/spectrum` and `/api/v1/public/spectra` do not include these fields. The worked-example evaluations below exclude ID 2 (clip fraction 0.77).

S3 Downsampling: viewer configuration, equal-budget variant and full results

The algorithm does not detect peaks itself. The web viewer uses a TypeScript implementation of the same rule (`core/client/src/workers/downsampleWorker.ts`) on the full stored range (164.35–878.26 nm, 2,094 channels) with ratio 0.1, i.e. $T = \min(\max(3, \lfloor 209.4 \rfloor), n) = 209$, and passes as $\mathcal{T}$ the 32 wavelengths of its built-in list (`core/client/src/utils/spectralUtils.ts`); the 33 lines of the database table `nist_lines` are drawn as overlays and are not downsampling targets. In the evaluation of main-text Section 4.3 we pass the detected peaks of the spectrum ("LTTB+Peaks"), for which peak retention is 100% by construction.

Equal-budget evaluation variant (not part of the production code). To compare LTTB+Peaks with the other methods at the same output size, `revision_v2/analysis.py` reduces the LTTB budget from $t = T$ in steps of one until $|S_{\text{LTTB}}(t) \cup P| \leq T$, where P is the set of detected-peak channels, and outputs that union. The union removes duplicates, so an LTTB channel that is also a peak counts once. The first and last channels belong to every $S_{\text{LTTB}}(t)$ and are therefore always kept. Because each change of t moves all bucket boundaries, the union does not shrink by exactly one per step, and the first admissible t can give fewer than T points (129–130 at $k = 4$). The variant is not computed when $|P| \geq T$ (ID 4 at $k = 5$: 73 peaks, $T = 65$).

Two caveats govern the interpretation. First, LTTB+Peaks keeps 100% of detected peaks because the detected peaks are its input; this is a property of the construction, not an empirical finding. Its cost is the measurable quantity: when it is allowed T points plus the peaks it outputs 147–167 points (8.7–9.9%), and at an equal 130-point budget its RMSE exceeds that of LTTB by 9.0–22.0 a.u. (11.7–31.0%). Second, reference wavelengths as targets preserve the reference channels, not the peaks. With the 24-line analysis list the server implementation keeps 47.5–61.9% of detected peaks at $k = 4$ (38.1–44.1% at $k = 5$). The web viewer as configured, evaluated by running its TypeScript function under Node.js

(`revision_v2/browser_downsampling.py`), keeps 55.9–71.4% at its default 209-point threshold with its 32-line list, against 50.8–69.0% for LTTB alone. For the same inputs, the TypeScript and Python implementations returned identical channels in all 24 tested configurations (three spectra, two ranges, two thresholds, with and without the viewer list). Across $k = 1$–5, LTTB alone keeps 85.7–97.3% of peaks at $k = 1$ and 35.6–38.1% at $k = 5$; LTTB+Peaks outputs 62.3–62.5% to 5.4–6.7% of the channels (main-text Figure 4). Uniform decimation has the lowest RMSE of the tested methods but discards most peaks, so no single method is best on both measures.

S4 Candidate annotation: per-spectrum detail and null-model design

Main-text Table 5 lists the coincidences for IDs 1, 3, 4; the per-peak table (observed and reference wavelength, signed offset $\lambda_{\mathrm{obs}} - \lambda_{\mathrm{ref}}$, prominence, all references within 0.5 and 1.1 nm) is in `D_per_line_candidates.csv`. For ID 3 the eight coincidences are near Fe I 274.95, 385.99, 495.76; Ca II 393.37, 396.85; Mg I 517.27; Na I 588.99/589.59; and O I 777.19 nm, with offsets from −0.09 to +0.48 nm (main-text Figure 5). Three of them have another reference within 1.1 nm (Ca II 393.37/Al I 394.40, Al I 396.15/Ca II 396.85, Na I 588.99/589.59) and cannot be assigned to one transition; the label does not imply that every listed species is present. Four further references have a detected peak 0.79–0.83 nm away (Al I 309.27, Si I 390.55, Al I 396.15, H I 656.28), outside the tolerance. The most prominent feature of ID 3, at 282.3 nm, has no line of the 24-line reference list within 1.1 nm; the full NIST database was not searched.

Null model. In each draw the whole 24-line list is shifted by one common offset s, $|s| \sim U(2, 20)$ nm with random sign, so the spacings between reference lines are preserved, and the matching rule is re-applied to the unchanged peaks. For a single spectrum the statistic is its number of matched peaks (10,000 draws, seed 20261021). For an archive-wide test, each draw applies the same shift to every spectrum, and the statistic is the sum of the per-spectrum counts (1,000 draws, seed 20261022). The wavelength-axis comparison uses the first 300 of these archive draws for both axes. Shifted lines that leave 220–800 nm cannot match; a variant that wraps them back into the range gave identical results, because no shift moves a line out of range. With k draws at or above the observed count among N, $p = (1 + k)/(1 + N)$. Main-text Table 5 gives N, k and p for every test; the same values are in `D_null_summary.csv`.

Only ID 3 is statistically significantly enriched under the specified null; IDs 1 and 4 and both archive-wide aggregates are not ($p \geq 0.062$) (main-text Figure 6). Labelling the same detected channels with the single-shot wavelength axis instead of the parent-table axis gives 1,386 archive coincidences against a null mean of 1,451.8 ($p = 0.72$; parent axis with the same 300 draws: $p = 0.10$). An aggregate excess, where present, says that the peak list and the reference list share more wavelengths than under the null; it does not establish that any individual assignment is correct. No line-ratio analysis is performed: ratios between lines depend on plasma temperature, density, self-absorption and instrument response, and tabulated transition probabilities alone do not predict observed intensity ratios.

A separate screening utility in the pipeline uses a fixed height threshold and is not used for any number in the manuscript or this supplement.

S5 API performance: protocols and earlier measurements

Earlier single-connection campaign (not repeated). Measured in July 2026 on an Intel Core i7-11850H/16 GB/Windows 11 host, 1,000 requests per configuration after 20 discarded, caching bypassed, Measurement ID 3 via `/api/v1/spectrum`: local server with co-located PostgreSQL, median 17.13 ms (P95 73.06, P99 88.60); local server with a remote (Supabase, WAN) database, median 296.27 ms (P95 471.91, P99 673.62); production deployment, median 1,056.37 ms (P95 1,462.01, P99 1,650.62; percentiles recomputed from the archived raw latencies in `latency_results/`). Each WAN configuration was measured in one session, so session-to-session and time-of-day variation are not characterised. The production deployment was re-measured in three sessions in this revision (below); the configuration with a remote database was not repeated.

Local concurrency sweep (this revision). A throwaway PostgreSQL 18 cluster holding the worked-example product, one uvicorn worker (Python 3.11.9), client and server on the same host over loopback, Redis not running (every request is a cache miss; `use_cache=false` also sent). Three sessions, each with a fresh server process; per session, workload and concurrency level, 20 warm-up requests were discarded and 400 measured (14,400 measured requests in total, 0 errors, 0 timeouts at a 10 s limit; `revision_v2/run_local_benchmark.py`). Throughput saturates at one worker, so latency at higher concurrency is dominated by queueing (Table S3). The query uses the measurement index and runs in about

1 ms once planner statistics exist; on a freshly loaded database without statistics the same query ran as a nested-loop join in 422 ms, so ingestion now ends with `ANALYZE`. These loopback results characterise the software on one host; they do not represent WAN latency or multi-worker deployments.

Workload	Conc.	P50	P95	P99	Req/s
2,094 points, 131 kB	1	19.9	51.4	64.9	41.7
	4	88.3	129.2	136.4	45.2
	32	717.7	977.5	1,181.3	43.7
1,684 points, 106 kB	1	16.7	46.1	58.2	49.2
	32	528.6	713.7	789.4	57.1
LTTB k=4, 130 points, 8.6 kB	1	8.6	18.0	34.7	96.4
	32	230.9	310.2	346.2	135.9

Table S3: Local loopback sweep, cache disabled: latency (ms) and throughput, medians over three sessions (all levels 1/4/16/32 and per-session values in `revision_v2/outputs/benchmark/summary.csv`).

Public deployment over the WAN (this revision). On 23 October 2026 (22:32–23:28 IST) the client host above sent requests over the internet to `https://apis.lunaratlas.in/api/v1/spectrum`. This deployment holds the legacy-ingested records, and ID 3 is identical to the revision output (main-text Listing 1). There were three sessions, 15 min apart. For each session, workload, cache condition and concurrency level (1, 4, 8), 10 warm-up requests were discarded and 100 measured: 3,600 measured requests, 0 errors, 0 timeouts at a 30 s limit (`revision_v2/run_wan_benchmark.py`). The cache conditions were `use_cache=false` and a Redis cache filled by the warm-up requests. Session medians agreed to within 352 ms for full spectra and 105 ms for downsampled responses (main-text Table 6). With the cache bypassed and one request at a time, the server reported about 530 ms of processing (`server_processing_ms`) for both workloads. The remaining 260–520 ms of the 790–1,050 ms median is network and transfer. For cache hits this field repeats the value stored with the cached response, so it is not reported. Concurrency was capped at 8 to limit load on the shared deployment, so its saturation point was not determined. The deployment's hardware and worker count are not characterised here, and all requests came from one client location.

Testing

The `pytest` suite runs the real stage-2 processor on synthetic L1 files (pairing, exclusion of unpaired plasma, identifier stability, pre-clamp clip fraction, background provenance, ambiguous flags), the server LTTB implementation (output size, endpoints, preserved targets), and formula-level checks of subtraction and clamping. An ingestion test (`tests/test_ingestion_db.py`) runs against a disposable PostgreSQL database and checks raw/clean identifier linkage, survival of the quality fields and idempotent re-ingestion. The GitHub Actions workflow runs the non-database tests for Python 3.10–3.12. The archive-level checks of main-text Section 4.1 are separate scripts in `revision_v2/`.